

ERC-5564

Stealth Addresses

Eddy Chen
Andre Fong
Arwin Fong

What is ERC-5564

A standard interface for non-interactive **stealth address** generation.

Demo

Pros/cons of using Crypto Mixers

- ✓ Completely breaks the link from previous wallet to new address
- ✓ Useful when there is large amounts of crypto
- ⦿ Can only move a set amount of ETH
- ⦿ Your funds could be mixed with malicious actors thus your funds could be flagged or frozen

Pros/cons of creating multiple wallets

- ✓ Easy to create new wallets
- Managing dozen private keys for each wallet
- Traceability

Pros/cons of using Stealth Addresses

- ✓ Non-interactive
- ✓ Recipient remains private when receiving assets
- ⦿ Problem: If a stealth address has ERC-20/ERC-721/etc. tokens, a user cannot transfer them from their stealth address since it has no ETH

<https://eips.ethereum.org/EIPS/eip-7702>

Stealth Meta-Addresses

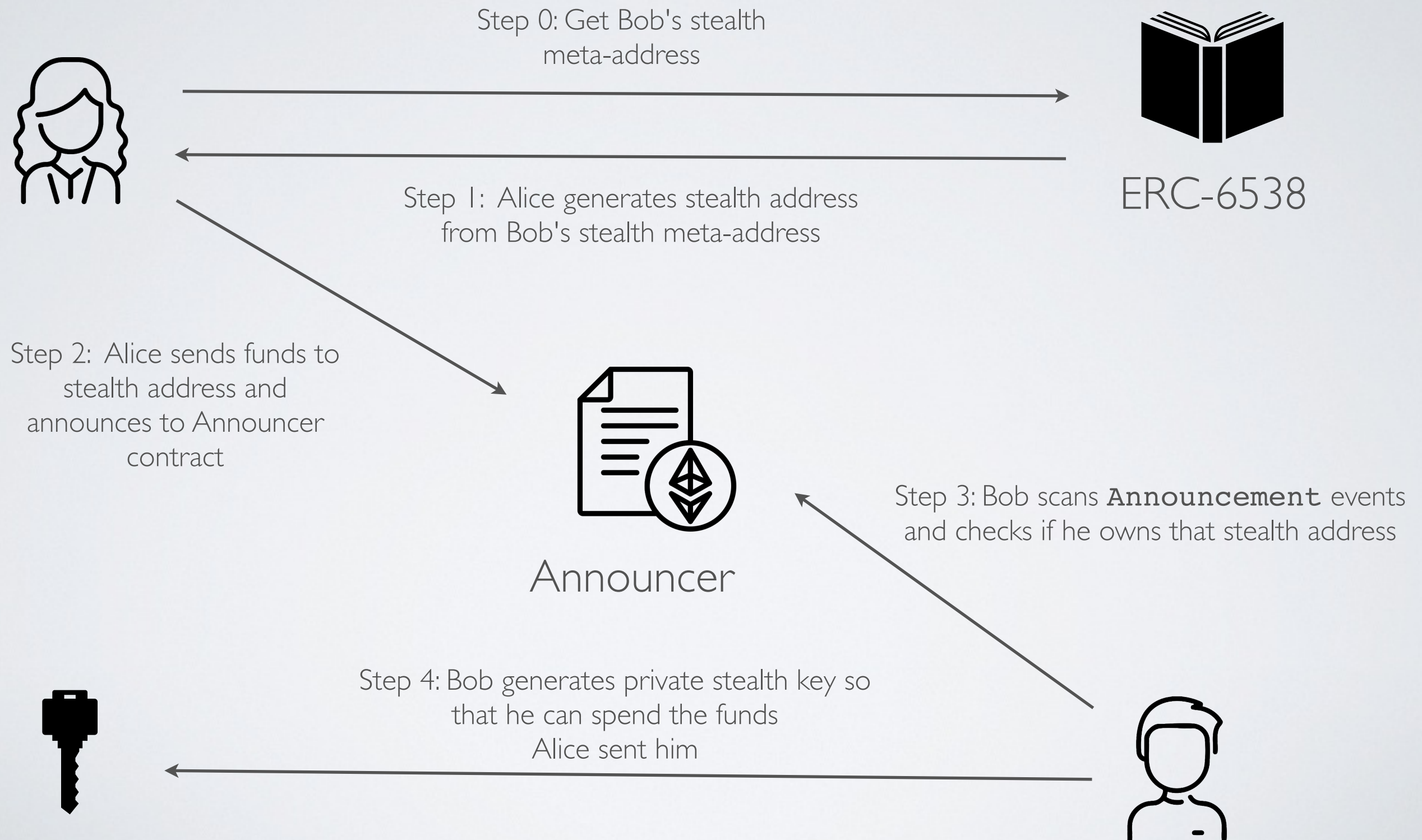


Stealth Meta-Addresses

How do we send a stealth meta-address without directly sharing it to someone?

✓ Use ERC-6538: Stealth Meta-Address Registry

Stealth Address



generateStealthAddress

1. Sender generates 32-byte entropy ephemeral private key and corresponding ephemeral public key ($p_{\text{ephemeral}}$ and $P_{\text{ephemeral}}$ respectively)
 2. $s = p_{\text{ephemeral}} * P_{\text{view}}$
 3. $S_h = s * G$
 4. $P_{\text{stealth}} = P_{\text{spend}} + S_h$
 5. Compute and return recipient's stealth address a_{stealth} as **pubkeyToAddress**(P_{stealth})
-
- ➔ Recipient publishes stealth meta-address which is used by the sender in **generateStealthAddress**
 - ➔ Generator point is a point on the SECP256k1 elliptic curve defined as $y^2 = x^3 + 7 \pmod{p}$, where $p = 2^{256} - 2^{32} - 977$

checkStealthAddress

1. $s = p_{\text{view}} * P_{\text{ephemeral}}$
2. $s_h = h(s)$
3. Extract most significant byte $s_h[0]$ in hashed shared secret for the view tag. If the view tag does not match the **Announcement** event's view tag skip, the steps below
4. $S_h = s_h * G$
5. $P_{\text{stealth}} = P_{\text{spend}} + S_h$
6. Compute recipient's stealth address a_{stealth} as **pubkeyToAddress**(P_{stealth})
7. Return **true** if stealth address of the announcement matches the derived stealth address, else return **false**

- ➔ Recipient has access to the viewing private key and the spending public key
- ➔ Recipient has access to **Announcement** events
- ➔ Generator point **G** is a point on the SECP256k1 elliptic curve defined as $y^2 = x^3 + 7 \pmod{p}$, where $p = 2^{256} - 2^{32} - 977$

computeStealthKey

1. $s = p_{\text{view}} * P_{\text{ephemeral}}$
2. $s_h = h(s)$
3. $p_{\text{stealth}} = p_{\text{spend}} + s_h$

➡ Recipient has access to the viewing private key and spending private key